

Lesson 2

CLI & Professional Environment

Code the Dream — Intro to Python

EXPLORE

Today's Session

EXPLORE

- What the terminal is and why it matters
- Essential CLI commands
- Absolute vs. relative paths
- Running scripts locally
- Git vs. GitHub

APPLY

- Navigate with the CLI
- Temperature converter
- First pull request

By the end: navigate your file system from the terminal, run Python scripts locally, and make your first Git commit and PR.

EXPLORE

What Is the Terminal?

The terminal is a **text interface for your computer**. Instead of clicking, you type commands.

GUI (what you're used to)

- Click folders to navigate
- Drag files to move them
- Double-click to run programs

Terminal (what developers use)

- `cd Documents` to navigate
- `mv file.py folder/` to move
- `python script.py` to run

EXPLORE

The Commands You'll Use Every Day

Command	What it does	Example
<code>pwd</code>	Print current location	<code>pwd</code>
<code>ls</code>	List files in current folder	<code>ls</code>
<code>cd</code>	Move into a folder	<code>cd Documents</code>
<code>cd ..</code>	Go up one level	<code>cd ..</code>
<code>mkdir</code>	Create a new folder	<code>mkdir my_project</code>
<code>touch</code>	Create a new empty file	<code>touch hello.py</code>

Press `Tab` while typing a name — the terminal will finish it for you.

EXPLORE

Absolute vs. Relative Paths

Absolute — full address from the root.
Works from anywhere.

```
cd /Users/jazmine/Documents/python_class
```

Relative — from your current location.
Shorter, but position-dependent.

```
cd python_class/working  
cd ..  
cd ~
```

Check for Understanding

You are in `~/Documents` . You run `cd ..` . Where are you now?

- A) Still in Documents — nothing changed
- B) Your home directory (`~`)
- C) The root of the file system
- D) You get an error

EXPLORE

Virtual Environments

A **virtual environment** keeps one project's packages separate from the rest of your computer.

```
python -m venv .venv          # create

source .venv/bin/activate      # activate (macOS/Linux)
source .venv/Scripts/activate # activate (Windows Git Bash)
```

When it's active, your prompt shows `(.venv)`. If you don't see that, run the activate command again.

EXPLORE

Running Python Scripts Locally

Navigate to your project folder, then run:

```
python hello.py
```

The terminal shows any `print()` output. This is called **stdout**.

The loop from now on: Write → Save (`Ctrl+S` / `Cmd+S`) → Run → Read → Fix → Repeat

Check for Understanding

You run `python my_script.py`. Nothing appears — no output, no error. What is the most likely explanation?

- A) Python isn't installed correctly
- B) The script ran but has no `print()` statements
- C) There is a bug in the code
- D) You need to activate your virtual environment first

EXPLORE

Git vs. GitHub

Git

- Software on your computer
- Tracks changes to files
- Works offline
- Stores the full history locally

GitHub

- A website (cloud service)
- Hosts Git repositories online
- Lets you share and collaborate
- Where you submit your work

EXPLORE

The Git Commit Cycle

Each time you make meaningful progress, save it with a commit.

```
git status
git add filename.py
git commit -m "describe the change"
git push origin assignment-2
```

Commit often — every commit is a point you can return to.

Check for Understanding

You edited a file and want to save your changes to GitHub. What is the correct order?

- A) `git push` → `git add` → `git commit`
- B) `git commit` → `git add` → `git push`
- C) `git add` → `git commit` → `git push`
- D) `git push` → `git commit` → `git add`

Now: Apply

Let's use the terminal and run some code.

APPLY

This Week's Assignment

Part 1: Warmup Exercises

1. Print "Python is working!" and record the terminal command + output
2. Navigate to your folder using only CLI commands
3. Make a meaningful commit and record `git log --oneline`
4. Write a script with a deliberate bug, read the error, fix it

Part 2: Mini-Project — Temperature Converter

Submit a **pull request** from `assignment-2` into `main` — not just your repo link.

APPLY

Code-Along: Navigate & Run

```
pwd                                # confirm where you are
cd ~/python-intro-homework/week-2/assignment-2
touch warmup1.py
code warmup1.py
```

```
# warmup1.py
print("Python is working!")
```

```
python warmup1.py
```

APPLY

Code-Along: Temperature Converter

```
fahrenheit = float(input("Enter a temperature in Fahrenheit: "))  
celsius = (fahrenheit - 32) * 5 / 9  
print(f"{fahrenheit}°F is {celsius:.1f}°C.")
```

Try it: Enter `72` . Expected: `72.0°F is 22.2°C.`

APPLY

Submitting Your First PR

```
git checkout main
git pull origin main
git checkout -b assignment-2

git add warmup1.py mini_project.py
git commit -m "add warmup exercises and temperature converter"
git push origin assignment-2
```

Then on GitHub: open your repo → **Compare & pull request** → set base to `main` → submit.

Submit the **PR link**, not your repo homepage:

```
github.com/your-username/python-intro-homework/pull/1
```

APPLY

Extension: Read an Error Message

Write a script with a **deliberate bug**, run it, read the error, then fix it.

```
year = input("Year you were born: ")  
age = 2026 - year
```

In the file, paste the full error message and explain what caused it and how you fixed it.

EXPLORE

Wrap-Up

This week you learned to:

- Navigate your file system from the terminal
- Run Python scripts locally with `python script.py`
- Keep a virtual environment active while working
- Stage, commit, and push code with Git

Before next session: Complete all four warmup exercises and the temperature converter. Submit your PR link + video reflection in CTD Learns. Post questions in **#[class-name]-discussion** on Slack.